The World Finals Mindset

Format, problem-set archetypes, 5-hour strategy, and what past finals teach us

Learning goals

- Know the World Finals format precisely: what is fixed, what varies year to year
- Recognise the **archetypes** that recur in WF problem sets
- Run a 5-hour contest as a team with **one** computer
- Estimate where your team sits on the difficulty curve and plan to move up
- Use past finals as training material: how to read, solve, and post-mortem them
- Set up the training plan for the rest of this track

The format (stable facts)

Item	World Finals
Team	3 contestants, 1 coach
Machine	**1 computer** per team
Duration	**5 hours**
Problems	usually ~11–13, *not* sorted by difficulty
Reference	printed team reference document (historically ≤ 25 pages)
Ranking	problems solved, then penalty time
Scoreboard	frozen for the final hour

Always read the current year's rules and environment page: languages, page limits and tooling change.

Scoring, exactly

- Primary key: **number of problems solved** (more is better)
- Secondary key: **penalty time** (less is better)
  - for every *solved* problem: minutes from contest start to the accepted run
  - plus **20 minutes per rejected run** on that problem before acceptance
- Rejected runs on problems you never solve cost **nothing**

Consequence: one extra solve beats any amount of penalty. A wrong submission at minute 280 is free if the problem never gets accepted, and costs 20 minutes if it does.

What "WF difficulty" means (rough guide)

Tier	Typical share of set	Who solves it
Easy	2–3 problems	almost every team
Medium	3–4 problems	separates medal zone from the field
Hard	3–4 problems	decides the medals
Very hard	1–2 problems	often solved by few or no teams

- Medal teams solve most of the set; the gap between rank 1 and rank 12 is often **one problem or penalty**
- The easy problems at WF are roughly regional-medium

Archetype 1: the "obvious once seen" easy

- Short statement, small model, the trick is to *not overthink*
- WF 2017 *Secret Chamber at Mount Rushmore*: letter translations → reachability / transitive closure on 26 nodes
- WF 2016 *Ceiling Function*: build BSTs, count distinct shapes
- WF 2019 *Circular DNA*: prefix balance over rotations

Team rule: whoever reads an easy problem **codes it immediately**. Speed on these is pure penalty.

Archetype 2: binary search on the hidden quantity

- WF 2017 *Need for Speed*: unknown speedometer offset `c`; total time is monotone in `c` → bisect
- Recognise: "find the parameter that makes this equation hold" + monotonicity
- Pitfalls: domain lower bound (`c > -min s_i`), iteration count with doubles, relative vs absolute error

```
lo = -min(s) + tiny, hi = big
repeat 100 times: mid = (lo+hi)/2; if f(mid) > t then lo = mid else hi = mid
```

Archetype 3: classical algorithm, disguised

- WF 2016 *Branch Assignment*: grouping costs → divide-and-conquer DP optimisation
- WF 2017 *Money for Nothing*: maximise area between two point sets → monotone structure, D&C on dominance
- WF 2015 *Catering*: assignment with capacity → min-cost flow
- WF 2015 *Tile Cutting*: count via a convolution of divisor counts → **FFT/NTT**

The modelling step is the problem. The algorithm is in your notebook.

Archetype 4: geometry and precision

- At least one geometry problem appears in essentially every WF set
- Many are "hard" because of **cases and precision**, not ideas
- WF 2017 *Airport Construction*: longest segment inside a simple polygon
- WF 2018 *Panda Preserve*: Voronoi-flavoured covering of a polygon

Decide early: integer-exact (cross products in `long long` / `__int128`) or `long double` with explicit epsilon policy (Lecture 6).

Archetype 5: the long implementation

- Simulation or parsing with many rules; correctness is the whole challenge
- WF 2019 *Karel the Robot*: interpret a tiny language, detect infinite loops
- WF 2017 *Visual Python++*: match rectangle corners, check nesting

Strategy: assign to your most careful implementer, write it on paper first, and give them a long uninterrupted block at the keyboard.

Archetype 6: the deep problem

- Needs a technique from this track (LP duality, link-cut trees, polynomial algebra, slope trick …) **plus** an insight
- WF 2018 *Conquer the World*: tree min-cost-flow structure, solved with mergeable heaps / greedy with regret
- WF 2017 *Son of Pipe Stream*: flow + continuous optimisation

You win medals by solving **one** of these that the teams around you do not.

The 5-hour timeline (a template, not a law)

Time	Activity
0:00–0:15	split the set three ways, read everything, mark difficulty
0:15–1:00	code the easies in order of certainty; others think
1:00–3:00	medium tier; pair on the hardest viable one
3:00–4:00	commit to the last 1–2 problems you can finish
4:00–5:00	frozen board: debug, fix WAs, no new starts after ~4:30 unless tiny

Revisit the plan at 1:00, 2:30 and 4:00 as a team.

Reading the scoreboard

- Problems are usually lettered in **alphabetical order of title**, not difficulty order
- The live scoreboard is a free difficulty oracle for the first four hours
  - many teams solved X early → X is easy, you are missing something if stuck
  - nobody has touched Y at 2:00 → Y is probably very hard or a trap
- Don't follow blindly: strong teams sometimes start with long problems
- After the freeze you only see your own results

One computer, three people

- Keyboard time is the bottleneck: **never think at the keyboard**
- Code on paper (pseudo-code + edge cases) while someone else types
- Debug on **printouts**, not on screen, while the next person codes
- Hand-offs are explicit: "I need the machine for ~15 min for problem C"
- Keep a shared problem sheet: status, owner, idea, test cases

Roles that work at WF

Role	Responsibilities
Driver	fastest clean coder; types most easies and most long implementations
Thinker	owns the hardest viable problem, proves things, writes pseudo-code
Verifier	reads every statement, hunts edge cases, writes tests and brute forces

Roles rotate per problem. What must be fixed is **who decides** when there is a conflict over the machine.

Debugging under freeze pressure

1. Re-read the statement *out loud* to a teammate (limits, output format, multiple tests?)
2. Check overflow, uninitialised arrays, reset between test cases
3. Write a brute force + random generator; stress test (3 lines of shell)
4. Construct extreme tests: n = 1, all equal, maximum sizes, degenerate geometry
5. Only then rewrite

A 20-minute penalty is cheaper than 40 minutes of blind staring.

Stress test in 30 seconds

```
for i in $(seq 1 1000); do
  ./gen $i 8 > in
  ./fast < in > out1
  ./brute < in > out2
  if ! cmp -s out1 out2; then echo "diff on seed $i"; cat in; break; fi
done
```

- Keep `gen.cpp` and `brute.cpp` templates in the notebook
- Generators take a **seed** argument so failures are reproducible
- Shrink `n` until the counterexample is human-sized

Estimating difficulty from the statement

- **Small constraints** ($n \le 20$, $n \le 500$) are hints: bitmask, cubic, flows
- **Strange constraints** ($n \le 10^5$ but $k \le 50$) say which parameter is in the exponent
- **Output mod 998244353** suggests NTT / polynomial work
- **Real-valued output with 10^{-6} error** suggests binary search, geometry, or LP
- **"Answer exists"/"print any"** suggests construction or matching

Training with past finals

- Full past sets: practise as **5-hour virtual contests**, 3 people, 1 machine
- Upsolve every problem solved by ≥ 5 teams; read the editorial / judges' solution for the rest
- Keep a log: problem → archetype → technique → why we missed it
- Mix in Codeforces Div1 D–F and AtCoder AGC for technique depth
- One full simulation every 1–2 weeks; technique lectures in between

Post-mortem template

Question	Example answer
What did we solve, and when?	A 0:12, F 0:31, C 1:05 …
Where did the time go?	50 min on D: wrong model, should have abandoned at 0:30
Wrong submissions?	2× on F: `int` overflow, missed n = 1
What would the top teams have done?	started G, which was a known technique
Notebook gap?	no half-plane intersection → add it

The mental game

- Expect to be stuck on most problems most of the time: that is normal at WF
- The freeze makes everyone look worse to themselves; ignore what you cannot see
- A dead problem is a sunk cost; rotate it to a fresh pair of eyes
- Keep eating, drinking, and talking; silence in a team is a warning sign
- After 4:30, small fixes beat new starts

The rest of this track

#	Lecture
2	Polynomials: NTT applications, inverse/log/exp, Berlekamp–Massey, Lagrange
3	Hungarian, blossom, LP duality & simplex, flows with demands
4	Link-cut trees, DSU on tree, virtual trees, rerooting
5	CDQ, parallel binary search, offline dynamic connectivity, Mo with updates
6–9	geometry, combinatorics, strings, hard DP
10	Team reference document & full WF simulation

Practice for this week

- Full virtual contest: a past WF set from the Kattis archive, 5 hours, 3 people, 1 PC
- WF 2017 *Secret Chamber at Mount Rushmore*, *Need for Speed* (Kattis)
- WF 2016 *Swap Space* (exchange-argument greedy, Kattis)
- WF 2015 *Tile Cutting* (FFT, Kattis) — preview of Lecture 2
- Codeforces 1305 F *Kuroni and the Punishment* (randomisation + insight)

Summary

- WF = 3 people, 1 computer, 5 hours, ~11–13 unsorted problems, penalty 20 min / rejection
- Problems fall into recognisable archetypes; most of the difficulty is **modelling**
- Win easies on speed, mediums on correctness, hards on one deep insight
- Keyboard time is the scarce resource: plan, print, and pair
- Train with full simulations and disciplined post-mortems